DESARROLLO WEB CON BLAZOR

Espacio de Definición Institucional

UNIDAD 1

Fundamentos de Blazor y Razor Components

Clases	1 a 8 (Clases 1–8 del cronograma)
Período	16 de marzo – 18 de mayo de 2026
Horas	8 clases × 2 horas = 16 horas reloj
Bibliografía	Web Development with Blazor – 3rd Ed., Cap. 1–5, 8

Introducción a la Unidad

Esta unidad inicia al estudiante en el desarrollo de aplicaciones web modernas con el framework Blazor de Microsoft. Se parte del modelo cliente-servidor clásico para llegar, de forma gradual y práctica, a la construcción de aplicaciones basadas en componentes Razor.

A lo largo de las ocho clases el alumno configurará su entorno de desarrollo, creará su primer proyecto Blazor, comprenderá la sintaxis Razor, desarrollará componentes reutilizables con parámetros, implementará binding de datos bidireccional, establecerá comunicación entre componentes mediante EventCallback, configurará el sistema de routing y construirá formularios con validación.

Al finalizar la unidad se desarrollará un mini-proyecto integrador: un Gestor de Tareas personal que consolida todos los conceptos trabajados.

Objetivos de Aprendizaje

- Comprender la arquitectura cliente-servidor y el lugar de Blazor en el ecosistema .NET.
- Configurar correctamente el entorno de desarrollo (Visual Studio / VS Code, SDK .NET 8).
- Crear y estructurar proyectos Blazor Server y Blazor WebAssembly.
- Dominar la sintaxis Razor: directivas, expresiones, bloques de código @code.
- Desarrollar componentes reutilizables con parámetros tipados y CascadingParameters.
- Implementar binding unidireccional (@bind) y bidireccional con ValueChanged.
- Manejar eventos del DOM y eventos de componente con EventCallback<T>.
- Comunicar componentes padres e hijos mediante parámetros y callbacks.
- Configurar routing con @page, parámetros de ruta y NavigationManager.
- Construir formularios con EditForm, DataAnnotationsValidator y mensajes de validación.
- Gestionar el ciclo de vida de un componente (OnInitialized, OnParametersSet, etc.).

Clase 1 – Arquitectura Web y Modelo Cliente-Servidor

Fecha	Clase	Tema
16 de marzo de 2026	Clase 1	Introducción a Blazor

Marco Teórico

La arquitectura cliente-servidor es el modelo fundamental de la web. El cliente (navegador) realiza solicitudes HTTP a un servidor que procesa la lógica y devuelve una respuesta. Blazor extiende este modelo integrando C# y .NET en el pipeline de renderizado.

Existen dos modalidades principales en Blazor:

- Blazor Server: el renderizado ocurre en el servidor; la UI se sincroniza con el cliente a través de SignalR.
- Blazor WebAssembly (WASM): el runtime .NET se ejecuta directamente en el navegador mediante WebAssembly.

> Diferencia clave: Blazor Server requiere conexión permanente; Blazor WASM descarga el runtime al navegador y puede funcionar offline. En este curso se trabaja con Blazor Server.

Conceptos Clave

- Protocolo HTTP/HTTPS y ciclo request-response
- DOM (Document Object Model) y Virtual DOM
- Componentes como unidad de composición de UI
- SignalR: comunicación bidireccional en tiempo real
- Razor: sintaxis de plantillas que combina HTML con C#

Diagrama: Arquitectura Blazor Server

```
Navegador (cliente)
     |
SignalR WebSocket
     |
ASP.NET Core (servidor)
     |
Componentes Razor (.razor)
     |
Árbol de renderizado → Diff → Delta UI → Navegador
```

Ejemplo de Código – Primer Componente Razor

```
@page "/contador"

<h3>Mi Primer Contador</h3>
<p>Valor actual: @contador</p>
<button @onclick="Incrementar" class="btn btn-primary">+1</button>
<button @onclick="Decrementar" class="btn btn-danger">-1</button>

@code {
    private int contador = 0;
```

```
    void Incrementar() => contador++;
    void Decrementar() => contador--;
}
```

Laboratorio Guiado

1. Instalar .NET 8 SDK y verificar con dotnet --version.
2. Crear solución: dotnet new blazorserver -o GestorTareas
3. Explorar la estructura de carpetas: Pages/, Shared/, wwwroot/, Program.cs.
4. Ejecutar la aplicación y navegar a /counter.
5. Modificar el componente para que muestre el doble del contador.

Ejercicios Propuestos

- Agregar un botón 'Reiniciar' que lleve el contador a cero.
- Crear una nueva página /bienvenida que muestre nombre y fecha actual con @DateTime.Now.
- Investigar: ¿qué diferencia hay entre @page y @layout en Razor?

Lectura en Bibliografía Oficial

Web Development with Blazor – 3rd Ed.
Capítulo 1: Introduction to Blazor
Capítulo 2: Building Your First Blazor Application

Clase 2 – Razor Components: Estructura y Ciclo de Vida

Fecha	Clase	Tema
30 de marzo de 2026	Clase 2	Razor Components

Marco Teórico

Un componente Razor es un archivo .razor que combina HTML y C# en una única unidad de UI. Cada componente posee un ciclo de vida bien definido que el desarrollador puede interceptar para inicializar datos, responder a cambios de parámetros o liberar recursos.

Los métodos del ciclo de vida más relevantes son:

- OnInitialized / OnInitializedAsync: se ejecuta una sola vez al montar el componente.
- OnParametersSet / OnParametersSetAsync: se ejecuta cada vez que los parámetros cambian.
- OnAfterRender / OnAfterRenderAsync: se ejecuta tras cada renderizado; útil para JS interop.
- Dispose / DisposeAsync: permite liberar suscripciones o recursos al desmontar.

Conceptos Clave

- Archivo .razor = HTML + @code + directivas
- Code-behind: separar lógica en archivo .razor.cs
- Parámetros: [Parameter] y [CascadingParameter]
- Renderizado condicional: @if, @switch, @foreach
- StateHasChanged() y cuándo invocarlo

Ejemplo de Código – Ciclo de Vida

```
@page "/demo-ciclo"
@implements IDisposable

<h3>Demo Ciclo de Vida</h3>
<p>Estado: @mensaje</p>

@code {
    private string mensaje = "No inicializado";

    protected override void OnInitialized()
    {
        mensaje = $"Iniciado a las {DateTime.Now:HH:mm:ss}";
    }

    protected override async Task OnInitializedAsync()
    {
        await Task.Delay(500); // simula carga asíncrona
        mensaje += " | Datos cargados";
    }

    public void Dispose()
    {
```

```
        // liberar recursos, cancelar suscripciones
    }
}
```

Laboratorio Guiado

6. Crear componente Tarjeta.razor con parámetros Titulo y Descripcion.
7. Mostrar cinco tarjetas desde Index.razor usando @foreach.
8. Agregar método OnInitializedAsync que simule carga con await Task.Delay(1000).
9. Mostrar un spinner mientras carga usando variable isLoading.

Ejercicios Propuestos

- Agregar parámetro Color (string) a Tarjeta.razor y aplicarlo al estilo del borde.
- Implementar IDisposable y registrar en consola cuándo se destruye el componente.
- Crear componente Alerta.razor que acepte parámetro Tipo ('success'|'warning'|'danger').

Lectura en Bibliografía Oficial

Web Development with Blazor – 3rd Ed.
Capítulo 3: Understanding Razor Components

Clase 3 – Data Binding y Eventos

Fecha	Clase	Tema
6 de abril de 2026	Clase 3	Binding y eventos

Marco Teórico

El binding en Blazor permite sincronizar el estado de C# con la UI de forma declarativa. El binding unidireccional (@variable) renderiza el valor cada vez que cambia. El binding bidireccional (@bind) equivale a suscribirse al evento de cambio y actualizar la variable automáticamente.

Los eventos del DOM se manejan con directivas @on{event}. Blazor soporta lambda y métodos, con variantes síncronas y asíncronas. Es posible pasar argumentos del evento (MouseEventArgs, ChangeEventArgs, etc.).

Conceptos Clave

- @bind como azúcar sintáctico de Value + ValueChanged
- @bind:event para elegir el evento disparador (oninput vs onchange)
- @onclick, @oninput, @onchange, @onkeydown con sus EventArgs
- Prevenir comportamiento por defecto: @onclick:preventDefault
- Stop propagation: @onclick:stopPropagation

Ejemplo de Código – Binding Bidireccional

```
<h3>Calculadora de IMC</h3>

<label>Peso (kg): <input type="number" @bind="peso" @bind:event="oninput"
/></label>
<label>Altura (m): <input type="number" @bind="altura" @bind:event="oninput"
/></label>

@if (altura > 0)
{
    <p>IMC: @(Math.Round(peso / (altura * altura), 2))</p>
}

@code {
    private double peso = 70;
    private double altura = 1.75;
}
```

Laboratorio Guiado

10. Crear formulario de registro con campos nombre, email y contraseña con @bind.
11. Mostrar un resumen en tiempo real debajo del formulario mientras el usuario escribe.
12. Agregar botón 'Limpiar' que reinicie todos los campos a cadena vacía.
13. Validar que la contraseña tenga al menos 8 caracteres y mostrar aviso.

Ejercicios Propuestos

- Agregar campo fecha de nacimiento y calcular la edad en tiempo real.

- Crear un selector de color (<input type="color">) y aplicar ese color al texto de resumen.
- Implementar contador de caracteres para el campo nombre (máximo 50).

Lectura en Bibliografía Oficial

Web Development with Blazor – 3rd Ed.
Capítulo 3: Understanding Razor Components (sección Data Binding)

Clase 4 – Comunicación entre Componentes

Fecha	Clase	Tema
13 de abril de 2026	Clase 4	Comunicación entre componentes

Marco Teórico

En una arquitectura de componentes, la información fluye de padres a hijos mediante parámetros ([Parameter]) y de hijos a padres mediante EventCallback<T>. Para estados compartidos entre componentes no relacionados directamente se utiliza un servicio de estado inyectado (patrón State Service).

CascadingParameter permite pasar un valor a todo un árbol de componentes sin necesidad de declararlo en cada nivel intermedio.

Conceptos Clave

- Flujo de datos unidireccional: padre → hijo (props down, events up)
- [Parameter] y [CascadingParameter] con CascadingValue
- EventCallback<T> y la diferencia con Action<T>
- ChildContent: RenderFragment para contenido proyectado
- Patrón State Service con IStateService registrado en DI

Ejemplo de Código – EventCallback y Parámetros

```
// TareaItem.razor
<div class="card mb-2">
    <div class="card-body d-flex justify-content-between">
        <span>@Tarea.Titulo</span>
        <button @onclick="EliminarClick" class="btn btn-sm btn-
danger">Eliminar</button>
    </div>
</div>

@code {
    [Parameter] public TareaModel Tarea { get; set; } = default!;
    [Parameter] public EventCallback<int> OnEliminar { get; set; }

    private async Task EliminarClick()
        => await OnEliminar.InvokeAsync(Tarea.Id);
}

// ListaTareas.razor (padre)
<TareaItem Tarea="t" OnEliminar="EliminarTarea" />

@code {
    private void EliminarTarea(int id)
        => tareas.RemoveAll(t => t.Id == id);
}
```

Laboratorio Guiado

14. Crear modelo TareaModel con Id, Titulo, Completada.

15. Desarrollar componente TareaItem.razor con EventCallbacks OnCompletar y OnEliminar.
16. Desarrollar ListaTareas.razor que instancie múltiples TareaItem y gestione la lista.
17. Crear CascadingValue con tema visual ('claro'/'oscuro') y leer desde TareaItem.

Ejercicios Propuestos

- Agregar EventCallback OnEditar que pase el Id de la tarea a editar al componente padre.
- Crear componente Contador global con State Service; incrementar desde dos páginas distintas.
- Usar RenderFragment para permitir contenido personalizado dentro de TareaItem.

Web Development with Blazor – 3rd Ed.
Capítulo 3: Understanding Razor Components (Component Parameters)

Clase 5 – Routing y Navegación

Fecha	Clase	Tema
20 de abril de 2026	Clase 5	Routing y navegación

Marco Teórico

Blazor implementa un router del lado del cliente. La directiva @page define la ruta de un componente. El sistema soporta rutas con parámetros tipados, rutas opcionales y restricciones de tipo. NavigationManager expone métodos para navegar programáticamente.

Conceptos Clave

- @page "/ruta" y múltiples directivas @page en un mismo componente
- Parámetros de ruta: @page "/tarea/{Id:int}" con [Parameter] public int Id
- Rutas opcionales con {Id:int?} y valor predeterminado
- NavigationManager: NavigateTo(), Uri, ToAbsoluteUri()
- NavLink y NavLinkMatch.All vs NavLinkMatch.Prefix
- Layouts: MainLayout.razor y @layout personalizado

Ejemplo de Código – Routing con Parámetros

```
@page "/tareas"
@page "/tareas/{FiltroEstado}"
@inject NavigationManager Nav

<h3>Tareas @(FiltroEstado ?? "todas")</h3>
<NavLink href="/tareas/pendiente">Pendientes</NavLink> |
<NavLink href="/tareas/completada">Completadas</NavLink>

@code {
    [Parameter] public string? FiltroEstado { get; set; }

    void IrADetalle(int id) => Nav.NavigateTo($"/tarea/{id}");
}
```

Laboratorio Guiado

18. Agregar al proyecto rutas /tareas, /tareas/{estado} y /tarea/{id:int}.
19. Crear página detalle que cargue la tarea por Id y muestre sus datos.
20. Implementar menú de navegación con NavLink y clase active automática.
21. Crear layout alternativo para la sección de administración.

Ejercicios Propuestos

- Agregar ruta /error con mensaje personalizado y redireccionar desde rutas inexistentes.
- Implementar breadcrumb dinámico usando NavigationManager.Uri.
- Proteger una ruta con AuthorizeView (solo visual, sin backend real).

Web Development with Blazor – 3rd Ed.
Capítulo 4: Routing and Navigation

Clase 6 – Formularios en Blazor

Fecha	Clase	Tema
27 de abril de 2026	Clase 6	Formularios

Marco Teórico

Blazor provee el componente EditForm vinculado a un modelo (EditContext) y una familia de componentes de entrada (InputText, InputNumber, InputDate, InputCheckbox, InputSelect) que se integran con el sistema de validación. El formulario puede reaccionar a los eventos OnValidSubmit y OnInvalidSubmit.

Conceptos Clave

- EditForm y el parámetro Model vs EditContext
- Componentes InputText, InputNumber, InputDate, InputCheckbox, InputSelect
- DataAnnotationsValidator y ValidationSummary
- ValidationMessage<TValue> para mensajes por campo
- OnValidSubmit, OnInvalidSubmit, OnSubmit

Ejemplo de Código – EditForm con Modelo

```
@page "/nueva-tarea"

<EditForm Model="nuevaTarea" OnValidSubmit="Guardar">
    <DataAnnotationsValidator />
    <ValidationSummary />

    <div class="mb-3">
        <label>Título</label>
        <InputText @bind-Value="nuevaTarea.Titulo" class="form-control" />
        <ValidationMessage For="() => nuevaTarea.Titulo" />
    </div>

    <div class="mb-3">
        <label>Prioridad</label>
        <InputSelect @bind-Value="nuevaTarea.Prioridad" class="form-select">
            <option value="Alta">Alta</option>
            <option value="Media">Media</option>
            <option value="Baja">Baja</option>
        </InputSelect>
    </div>

    <button type="submit" class="btn btn-success">Guardar</button>
</EditForm>

@code {
    private TareaModel nuevaTarea = new();
    private void Guardar() => /* lógica de guardado */ throw new
NotImplementedException();
}
```

Laboratorio Guiado

22. Crear página /nueva-tarea con EditForm y campos Titulo, FechaVencimiento, Prioridad.
23. Agregar Data Annotations al modelo: [Required], [MaxLength(100)], [FutureDate].
24. Al enviar el formulario válido, agregar la tarea a la lista y navegar a /tareas.

Ejercicios Propuestos

- Crear página /editar-tarea/{Id:int} que pre-cargue los datos en el formulario.
- Deshabilitar el botón submit mientras el formulario tiene errores.
- Agregar campo de archivo (InputFile) para adjuntar imagen a la tarea.

Web Development with Blazor – 3rd Ed.
Capítulo 8: Forms and Validation

Clase 7 – Validación Básica con Data Annotations

Fecha	Clase	Tema
4 de mayo de 2026	Clase 7	Validación básica

Marco Teórico

Data Annotations es el mecanismo estándar de .NET para declarar reglas de validación directamente en el modelo. Blazor integra este sistema mediante DataAnnotationsValidator. También es posible implementar IValidatableObject para validaciones cruzadas entre campos.

Conceptos Clave

- Atributos: [Required], [StringLength], [Range], [RegularExpression], [EmailAddress], [Url]
- [Compare] para confirmar contraseñas
- IValidatableObject para lógica de validación compleja
- CustomValidationAttribute: crear validaciones reutilizables
- EditContext.Validate() para disparar validación programáticamente

Ejemplo de Código – Modelo con Data Annotations

```
public class TareaModel
{
    public int Id { get; set; }

    [Required(ErrorMessage = "El título es obligatorio")]
    [StringLength(100, MinimumLength = 3,
        ErrorMessage = "Debe tener entre 3 y 100 caracteres")]
    public string Titulo { get; set; } = string.Empty;

    [Required(ErrorMessage = "Seleccione una prioridad")]
    public string Prioridad { get; set; } = "Media";

    [Required(ErrorMessage = "La fecha es obligatoria")]
    [DataType(DataType.Date)]
    public DateTime? FechaVencimiento { get; set; }

    public bool Completada { get; set; }
}
```

Laboratorio Guiado

25. Agregar todas las Data Annotations a TareaModel.
26. Crear CustomValidationAttribute FechaFuturaAttribute para validar que la fecha no sea pasada.
27. Implementar IValidatableObject: si prioridad es Alta, la fecha no puede ser mayor a 7 días.

Ejercicios Propuestos

- Crear modelo UsuarioRegistro con Email, Contraseña, ConfirmarContraseña y validar con [Compare].

- Mostrar un ícono de check verde / X roja junto a cada campo según su estado de validación.
- Crear validación de ReCaptcha simulado (checkbox 'No soy un robot').

Web Development with Blazor – 3rd Ed.
Capítulo 8: Forms and Validation

Clase 8 – Manejo de Estado en Blazor

Fecha	Clase	Tema
11 de mayo de 2026	Clase 8	Manejo de estado

Marco Teórico

El manejo de estado es uno de los desafíos centrales en aplicaciones SPA. Blazor no impone un patrón único; las opciones van desde variables locales del componente hasta servicios singleton de estado, pasando por CascadingValue y bibliotecas externas como Fluxor.

En Blazor Server el estado vive en el servidor (en memoria por circuito); en WASM vive en el cliente. Para persistir entre sesiones se usan ProtectedLocalStorage o APIs externas.

Conceptos Clave

- Estado local de componente: variables en @code
- Estado compartido: servicio registrado como Scoped o Singleton en DI
- Notificación de cambios: Action OnChange + StateHasChanged
- Persistencia: ProtectedLocalStorage (Blazor Server)
- Patrón simple de Flux: AppState → notifica → componentes se re-renderizan

Ejemplo de Código – State Service

```
// TareasState.cs
public class TareasState
{
    private readonly List<TareaModel> _tareas = new();
    public IReadOnlyList<TareaModel> Tareas => _tareas.AsReadOnly();
    public event Action? OnChange;

    public void AgregarTarea(TareaModel t)
    {
        _tareas.Add(t);
        NotificarCambio();
    }

    public void EliminarTarea(int id)
    {
        _tareas.RemoveAll(t => t.Id == id);
        NotificarCambio();
    }

    private void NotificarCambio() => OnChange?.Invoke();
}

// Program.cs
builder.Services.AddScoped<TareasState>();
```

Laboratorio Guiado

28. Crear TareasState como servicio Scoped y registrarlo en Program.cs.
29. Inyectar [Inject] TareasState en ListaTareas y NuevaTarea.

30. Suscribirse a OnChange en OnInitialized y desuscribirse en Dispose.
31. Verificar que agregar desde NuevaTarea actualiza ListaTareas automáticamente.

Ejercicios Propuestos

- Agregar a TareasState un método FiltrarPorPrioridad(string prioridad).
- Persisitir la lista de tareas en ProtectedLocalStorage para sobrevivir refrescos de página.
- Agregar estadísticas: total, completadas y pendientes, visibles en el layout principal.

Web Development with Blazor – 3rd Ed.
Capítulo 5: Managing State in Blazor

Mini Proyecto de la Unidad – Gestor de Tareas Personal

Desarrollar una aplicación Blazor Server funcional que implemente las siguientes características:

REQUERIMIENTOS DEL GESTOR DE TAREAS

Pantalla principal: listado de tareas con filtro por estado (todas / pendiente / completada)
Formulario de alta: crear tarea con título, descripción, prioridad y fecha de vencimiento
Formulario de edición: modificar cualquier campo de una tarea existente
Eliminar con confirmación modal (sin JS externo)
Marcar como completada / pendiente con toggle
Estadísticas en el header: total, completadas, pendientes
Validación completa con Data Annotations
Estado gestionado con TareasState (servicio Scoped)
Navegación con NavLink y rutas con parámetros
Diseño responsive con Bootstrap incluido en Blazor

Criterios de Evaluación

- Funcionalidad: todas las operaciones CRUD operativas.
- Componentes: al menos 4 componentes bien encapsulados y reutilizables.
- Validación: todos los campos con reglas y mensajes de error claros.
- Estado: uso correcto del State Service con notificación reactiva.
- Código limpio: nombres descriptivos, sin código duplicado.
- Entrega: proyecto en repositorio GitHub con README básico.

Bibliografía de la Unidad

Capítulo	Título	Clases
Cap. 1	Introduction to Blazor	1
Cap. 2	Building Your First Blazor Application	1–2
Cap. 3	Understanding Razor Components	2–4
Cap. 4	Routing and Navigation	5
Cap. 5	Managing State in Blazor	8
Cap. 8	Forms and Validation	6–7